

Assignment - 02

Title : Stack

Objective : To Study data structure and their implementation's using Stack.

Problem Statement : Implement Stack as an abstract data type using singly linked list and use this ADT for conversion of infix expression to postfix, prefix and evaluation of postfix and prefix expression.

Outcome : Implement abstract data type (ADT) and data structure's for given application's.

Theory :

- Stack :- It is a data structure which possesses last in first out property.

We can consider the e.g. Stack of Coins. The coins can be arranged one on another. When we add a new coin it is always placed on the previous coin and while removing the coin the recently placed coin can be removed.

Stack is a collection of element in which insertion and deletion of element is done by one end called top.

Operation's : 1) push

2) pop

1). Push : By this operation one can push element onto the stack. Before performing push we must check `stfull()` condition.

2). Pop : By this operation one can remove the element from stack. Before popping the element from stack, we should check `stempty()` condition.

If stack is empty, then we cannot pop and if stack is full we cannot push.

• Stack using linked organization / list :-

We can represent stack using array as well as linked list. The advantage of implementing stack using linked list is that we need not to worry about the size of the stack.

The structure for linked stack can be :-

```
{ struct stack
```

```
    int data;
```

```
    struct stack * next;
```

```
};
```

linked list consist of node and each node consist of data and the next field.

• Application of Stack :-

1). Expression Conversion.

2). Expression evaluation.

i) Expression Conversion :-

There are three types of expressions :

i) Infix ii) postfix iii) prefix

i) Infix :-

Infix expression = Operand 1 Operator Operand 2

e.g :- $(a + b)$

Most natural way of representing the expression

ii) Post Fix :-

Post Fix expression = Operand 1 Operand 2 Operator

e.g :- $ab +$

iii) Prefix :-

Prefix expression = Operator Operand 1 Operand 2

e.g :- $+ ab$

* Conversion from infix to postfix Expression :-

The Most important thing in the Conversion procedure is to know the priorities of the operators being used in it. This helps in determining the order of evaluation of operands

Operator	precedence in Stack	precedence in expression
$\wedge$	3	4
$*, /$	2	2
$+, -$	1	1
$($	0	4

Algorithm for Conversion of infix to postfix form;-

1. Read the expression from left to right
2. IF the input Symbol read is 'c' then push it onto Stack.
3. IF the input Symbol read is Operand then place it in postfix expression.
4. IF the input Symbol read is an operator then
 - a). Check if the precedence of the Operator which is in the Stack has greater precedence than the precedence of the operator read, if So then remove that symbol from Stack and placed it in the postfix expression.
 - b). Otherwise push the operator being read onto the Stack.
5. IF the input Symbol read is closing parenthesis ')' then pop all the operator from the Stack, place them in postfix expression till the opening parenthesis is not pop.

6. The opening & closing parenthesis should not be placed in the postfix expression.
7. Finally print the postfix expression.

Example :- $A * B + C \Rightarrow AB * C +$

Input	Stack	Output
A	Empty	A
*	*	AB
B	*	AB
+	Pop * & push +	AB * +
C	+	AB * C
	Empty	AB * C +

* Conversion from infix to prefix Expression :-

1. Scan the expression from right to left.
(Reverse the given infix expression).
2. If the input symbol read is operand then place it in prefix expression.
3. If the input symbol read is a closing parenthesis, then push it to the stack.
4. If the input symbol read is an opening parenthesis, pop the element in the stack until we find the corresponding closing parenthesis.
5. For operator same procedure to be performed as we have performed in postfix.
6. Finally reverse the expression again & print it.

e.g :- $A * B + C \Rightarrow + * ABC$.

2). Expression Evaluation :-

Algorithm for evaluation of postfix :

1. Read the Postfix expression from left to Right.
2. If the input Symbol read is an Operand then push it onto the stack.
3. If the Operator is read pop two Operand's and perform arithmetic Operation's if Operator is

'+' then result = Operand 1 + Operand 2

'-' then result = Operand 1 - Operand 2

'*' then result = Operand 1 * Operand 2

'/' then result = Operand 1 / Operand 2

4. push the result onto the Stack.

5. Repeat Step 1-4 till the postfix expression is not over.

For e.g :- Consider the postfix expression :-

1 2 3 + *

Step 1: Assume the array exp[] contain the input

exp					
1	2	3	+	*	\$

The \$ Symbol is used as an end Marker.

Read from first element of the array if it is Operand push it onto the Stack.

Stack

3	1	2	3	+	*
2					
1					

Step 2: If the operator is read pop two operands

Stack

3	→ op ₂
2	→ op ₁
1	

Perform $op_2 + op_1$

$$\text{Result} = 3 + 2 = 5$$

Push the result onto the Stack.

Now,

Stack

5	1	2	3	+	*	\$
1						

Read

Step 3: Again pop two operand's and perform the operation.

Stack

	5 * 1	1	2	3	+	*	\$
	⇒ 5						

Read

At, the last point of exp. the stack is empty and the input is read a \$. So here stop the evaluation procedure and return the result = 5

Test Case's :-

Test Case	Test Case description	Test data	Expected Result	Actual Result	pass/fail
1.	Conversion of infix expression to postfix	infix expression	Evaluation of postfix expression after input	Value of input get evaluated to postfix	pass
2.	Conversion of infix expression to prefix	infix expression	Evaluation of prefix exp. after input	Value of input get evaluated to Prefix	pass

```
#include<stdlib.h>
#include<iostream>
#include<string.h>
#define max 30
using namespace std;
struct node
{
    char data;
    struct node *next;
};
class stack
```

```
{
    node *top;
    char x;
public:
    stack()
    {
        top= NULL;
    }
    int empty()
    {
        if(top==NULL)
        {
            return(1);
        }
        else
        {
            return(0);
        }
    }
    void push(char x)
    {
        node *p;
```

```

    p=new node;
    p->data=x;
    p->next=top;
    top=p;
}
char pop()
{
    if(!empty())
    {
        node *p;
        p=new node;
        p=top;
        top=top->next;
        x=p->data;
        delete p;
        return(x);
    }
    else
    {
        cout<<"stack is empty"<<endl;
        return(0);
    }
}

```

```

    }
char pop1()
{
    if(!empty())
    {
        node *p;
        p=new node;
        p=top;
        //top=top->next;
        x=p->data;
        //delete p;
        return(x);
    }
    else
    {
        cout<<"stack is empty"<<endl;
        return(0);
    }
}
};

int precedence(char x);
void infix_to_prefix(char infix[],char prefix[]);

```

```
void infix_to_postfix(char infix[],char postfix[]);
```

```
void eval_prefix(char prefix[]);
```

```
void eval_postfix(char postfix[]);
```

```
int evaluate(char x,int op1,int op2);
```

```
int main()
```

```
{
```

```
    char infix[30],prefix[30],postfix[30];
```

```
    int op;
```

```
do{
```

```
    cout<<"\n\n\nMenu:: \n1)Infix to Prefix with Evaluation \n2)Infix to  
Postfix with Evaluation \n3)Quit \nEnter your choice:: ";
```

```
    cin>>op;
```

```
switch(op)
```

```
{
```

```
case 1:
```

```
    cout<<"\nEnter the infix expression::"<<endl;
```

```
    cin>>infix;
```

```
    infix_to_prefix(infix,prefix);
```

```
    cout<<"\nPrefix expression is "<<prefix<<endl;
```

```
    cout<<"\n\nEvaluation of Prefix expression";
```

```

    eval_prefix(prefix);
break;
case 2:
    cout<<"\nEnter the infix expression::"<<endl;
    cin>>infix;
    infix_to_postfix(infix,postfix);
    cout<<"\nPostfix expression is "<<postfix<<endl;
    cout<<"\n\nEvaluation of Postfix expression";
    eval_postfix(postfix);
break;
}
}while(op!=3);
return 0;
}

void infix_to_prefix(char infix[],char prefix[])
{
    int i,j;
    char temp,in1[30];
    cout<<"\n\n After step 1...\nEnter infix is...";
    for(i=0;i<=strlen(infix)-1;i++)
    {
        cout<<infix[i];
    }
}

```

```
}  
for(i=strlen(infix)-1,j=0;i>=0;i--,j++)  
in1[j]=infix[i];  
in1[j]='\0';
```

```
cout<<"\n\n After step 2...";  
for(i=0;i<=strlen(in1)-1;i++)  
{  
cout<<in1[i];  
}
```

```
for(i=0;in1[i]!='\0';i++)  
{  
if(in1[i]=='(')  
in1[i]=')';  
else  
if(in1[i]==')')  
in1[i]='(';  
}
```

```
cout<<"\n\n After step 3...";  
for(i=0;i<=strlen(in1)-1;i++)  
{
```

```

    cout<<in1[i];
}
cout<<"\n\n After step 4...";
cout<<"\n\n Prefix is...";
infix_to_postfix(in1,prefix);
for(i=0,j=strlen(prefix)-1;i<j;i++,j--)
{
    temp=prefix[i];
    prefix[i]=prefix[j];
    prefix[j]=temp;
}
}

```

```

void infix_to_postfix(char infix[],char postfix[])
{
    stack s;
    node *top;
    char x;
    int i,j;
    char token;

```

```
j=0;
for(i=0;infix[i]!='\0';i++)
{
    token=infix[i];
    if(isalnum(token))
    {
        postfix[j++]=token;
    }
    else if(token=='(')
    {
        s.push('(');
    }
    else if(token=='')
    {
        while((x=s.pop())!='(')
        {
            postfix[j++]=x;
        }
    }
    else
    {
        x=s.pop1();
```

```

while(precedence(token)<precedence(x)&& !s.empty())
{
    x=s.pop();
    postfix[j++]=x;
}
s.push(token);
}
while(!s.empty())
{
    x=s.pop();
    postfix[j++]=x;
}
postfix[j]='\0';
}

```

```

int precedence(char x)
{
    if(x=='(')
    {
        return(0);
    }
}

```

```

    }
    if(x=='+' || x=='-')
    {
        return(1);
    }
    if(x=='*' || x=='/' || x=='%')
    {
        return(2);
    }
    return(3);
}

```

```

void eval_prefix(char prefix[])
{
    stack s;
    char x;
    int op1,op2,val,i;
    for(i=strlen(prefix)-1;i>=0;i--)
    {
        x=prefix[i];
        if(isalpha(x))
        {

```

```

        cout<<"\nEnter the value of "<<x<<":";
        cin>>val;
        s.push(val);
    }
    else
    {
        op1=s.pop();
        op2=s.pop();
        val=evaluate(x,op1,op2);
        s.push(val);
    }
}
val=s.pop();
cout<<"\nValue of expression is "<<val;
}

```

```

void eval_postfix(char postfix[])
{
    stack s;
    char x;
    int op1,op2,val,i;
    for(i=0;postfix[i]!='\0';i++)

```

```

{
    x=postfix[i];
    if(isalpha(x))
    {
        cout<<"\nEnter the value of "<<x<<":";
        cin>>val;
        s.push(val);
    }
    else
    {
        op2=s.pop();
        op1=s.pop();
        val=evaluate(x,op1,op2);
        s.push(val);
    }
}
val=s.pop();
cout<<"\nValue of expression is "<<val;
}

```

```

int evaluate(char x,int op1,int op2)
{

```

```
if(x=='+')
return(op1+op2);
if(x=='-')
return(op1-op2);
if(x=='*')
return(op1*op2);
if(x=='/')
return(op1/op2);
if(x=='%')
return(op1%op2);
}
```